Complete Banking Auth Module & Session Security Architecture

1. Executive Problem Analysis

A critical vulnerability was identified where an unauthenticated session could access the dashboard in Auditor persona upon page reload. The root causes:

1. **Unverified LocalStorage Trust**: `useAuthStore.ts` read cached user data from `localStorage` if backend call threw an error or was slow, blindly setting `isAuthenticated: true`.
2. **Missing Cookie Invalidation**: Browser JavaScript cannot delete `HttpOnly` cookies (`aegis_access_token`, `aegis_refresh_token`). On `/logout`, the backend endpoint did not purge `aegis_user_persona`, and cookie deletion options did not strictly match setting parameters.
3. **No Strict Path-Based Routing / Navigation Guard**: Without route-level security, the app rendered operator tabs directly without route checks.
4. **Lack of Dedicated Customer Login / Portal Separation**: Customer accounts and operator accounts used the same login screen without partitioned routes.

2. Target Architecture

```
```

3. Detailed Component Plan

A. Backend (`src/AegisLedger.Api`)

1. `AuthEndpoints.cs`:
    - `POST /api/v1/auth/login`: Standard staff operator login (System Admin, Bank Teller, Compliance Auditor). Rejects Customer with explicit instructions to use Customer Portal.
    - `POST /api/v1/auth/customer/login`: Dedicated retail customer authentication endpoint. Grants Customer role claims and sets cookies.

- `POST /api/v1/auth/logout`: Purges all 4 cookies (`aegis_access_token`, `aegis_refresh_token`, `aegis_session_active`, `aegis_user_persona`) with matching `SameSite=Lax`, `Path=/`, and expired timestamp.
- `GET /api/v1/auth/me`: Verifies active session, returns role, persona, and user details.
- `POST /api/v1/auth/refresh`: Rotates JWT pair via Redis.

B. Frontend (`aegisledger-web-app`)

1. `proxy.ts` **(Client Navigation Proxy & Security Middleware)**:
   - Central URL and routing state manager (`/login`, `/customer/login`, `/customer/*`, `/`).
   - Browser history listener (`popstate`, `pushState`, `replaceState`).
   - Guard rules:
     - Public routes: `/login`, `/customer/login`.
     - Operator routes: `/` (and subtabs: `overview`, `accounts`, `transactions`, `holds`, `reconciliation`, `hangfire`).
     - Customer routes: `/customer/dashboard`, `/customer/accounts`, etc.
   - Redirects unauthorized roles and unauthenticated users immediately.
2. `useAuthStore.ts` **(Zero-Trust Session Store)**:
   - **Zero-trust initialization**: NEVER sets `isAuthenticated: true` without backend confirmation via `/api/v1/auth/me`.
   - On 401, attempts `/api/v1/auth/refresh`.
   - If both fail, clears all storage and forces `isAuthenticated: false`.
   - Recurring health & token verification polling interval (every 60s).
   - Dedicated `customerLogin()` and `operatorLogin()` methods.
3. `LoginPage.tsx` **&** `CustomerLoginPage.tsx`:
   - `/login`: Clean operator gateway for Tellers, Auditors, and Admins.
   - `/customer/login`: Dedicated Retail Customer login page with distinct styling, security seal, and customer presets.
4. `CustomerPortalView.tsx`:
   - Dedicated landing page for authenticated Retail Customers at `/customer/*`.
   - Displays customer account overview, pending balances, and placeholder for customer features.
5. `App.tsx` **Integration**:

- Mounts `proxy.ts` router.
- Shows "Authenticating..." security loader during verification.
- Renders Operator Portal, Customer Portal, or respective Login pages based on active route and verified role.